

ROLE

You are a Principal Software Engineer, Solution Architect, DevOps Engineer, Security Engineer, Product Engineer, and AI Integration Specialist with 15+ years of experience building scalable production-grade platforms.

Your task is to transform the existing MERN-based project "**Sahayog Sarthi**" into a production-ready, enterprise-grade, scalable platform.

DO NOT generate temporary fixes.

DO NOT create mock data unless explicitly required.

DO NOT leave TODO comments.

DO NOT create placeholder implementations.

Every implementation must be production quality.

PROJECT CONTEXT

Sahayog Sarthi is an Uber-like worker booking platform connecting customers with nearby service providers (electricians, plumbers, carpenters, technicians, etc.).

Current Stack:

Frontend:

- Next.js
- React
- Tailwind CSS

Backend:

- Node.js
- Express.js

Database:

- MongoDB Atlas
- Redis

Realtime:

- Socket.IO

Payments:

- Razorpay

Notifications:

- Telegram Bot

Deployment Target:

- AWS

EXISTING FEATURES

Already Implemented:

- JWT Authentication
- OTP Authentication Backend
- Worker Registration
- Geo-location Search
- Worker Discovery
- Booking Creation
- Live Tracking
- Socket.IO Chat
- Razorpay Orders
- Reviews & Ratings
- Telegram Notifications
- Admin Dashboard Skeleton

CRITICAL ISSUES TO FIX IMMEDIATELY

Payment Module

Replace the broken Payment model.

Current Payment.js incorrectly contains Express routes instead of a Mongoose schema.

Create:

Payment Schema

Fields:

- bookingId
- customerId
- workerId
- razorpayOrderId
- razorpayPaymentId
- amount
- platformFee
- workerAmount
- status
- paymentMethod
- transactionDate
- createdAt
- updatedAt

Add proper indexes.

Fix Redirect Bug

Current flow:

Payment page sends:

?payment=success&bookingId=123

Landing page expects:

?payment_success=true&booking_id=123

Standardize all payment success flows.

Create automated tests for this.

Fix Admin KYC Workflow

Current issues:

- Approve route broken
- Reject route missing
- Route parameters inconsistent

Implement:

PUT /workers/:id/approve

PUT /workers/:id/reject

PUT /workers/:id/block

PUT /workers/:id/unblock

Return standardized API responses.

Register Missing Routes

Implement:

GET /bookings/:id

GET /bookings/history

GET /bookings/chats/:id

GET /workers/admin/overview

POST /payments/withdraw

PUT /bookings/:id/cancel

PUT /bookings/:id/start

ARCHITECTURE UPGRADES

Refactor backend into Clean Architecture.

backend/

src/

modules/

auth/

worker/

booking/

payment/

review/

admin/

chat/

notification/

shared/

config/

middleware/

utils/

constants/

validators/

database/

models/

repositories/

services/

controllers/

routes/

Use:

- Controller Layer
- Service Layer
- Repository Layer

Strict separation of concerns.

API STANDARDS

Implement:

RFC-compliant REST APIs.

Standard response:

```
{  
  "success": true,  
  "message": "",  
  "data": {},  
  "meta": {}  
}
```

Standard error:

```
{  
  "success": false,  
  "error": {  
    "code": "",  
    "message": ""  
  }  
}
```

Use:

- express-validator
- Zod
- centralized error handling

SECURITY REQUIREMENTS

Implement:

Authentication:

- JWT Access Tokens
- Refresh Tokens
- Token Rotation

Security:

- Helmet
- CORS
- Rate Limiting
- Request Sanitization

- XSS Protection
- CSRF Protection
- Secure Cookies

Passwords:

- bcrypt

Secrets:

- Environment Variables Only

Never expose keys.

DATABASE IMPROVEMENTS

MongoDB:

Add indexes for:

Workers:

- location (2dsphere)
- serviceType
- rating

Bookings:

- customerId
- workerId
- status

Payments:

- bookingId
- workerId

Reviews:

- workerId

Implement:

- Transactions
 - Aggregation Pipelines
 - Soft Deletes
 - Audit Logs
-

REALTIME IMPROVEMENTS

Socket.IO

Implement:

Namespaces:

- customer
- worker
- admin

Features:

- typing indicators
- online presence
- read receipts
- delivery status
- reconnect handling

Persist messages in MongoDB.

Implement chat history retrieval.

PAYMENT SYSTEM

Razorpay Production Integration

Implement:

- webhook verification
- signature verification
- payment reconciliation
- idempotency keys
- failed payment recovery

Worker Wallet:

Create:

Wallet Schema

Transactions Schema

Withdrawal Requests Schema

Admin approval workflow.

BOOKING LIFECYCLE

Enforce state machine:

pending

→ accepted

→ in_progress

→ completed

→ paid

Alternative paths:

pending

→ cancelled

accepted

→ cancelled

Use strict validation.

Prevent invalid transitions.

NOTIFICATIONS

Implement:

FCM Push Notifications

Email Notifications

SMS Notifications

Notification Preferences

Notification Queue

Use:

BullMQ + Redis

FRONTEND REFACTOR

Upgrade to modern Next.js App Router.

Implement:

app/

(auth)

(customer)

(worker)

(admin)

Create:

Customer Pages:

- Profile
- Booking History
- Saved Addresses
- Payment History

Worker Pages:

- Earnings
- Availability
- Wallet
- KYC Status

Admin Pages:

- Dashboard
 - Analytics
 - Users
 - Workers
 - Complaints
 - Withdrawals
-

UI/UX REQUIREMENTS

Use:

- Tailwind CSS
- ShadCN UI
- Framer Motion

Requirements:

- Mobile First
 - Responsive
 - Accessibility (WCAG AA)
 - Dark Mode
 - Skeleton Loading
 - Optimistic Updates
-

AI FEATURES

Add AI-powered capabilities:

1. Smart Worker Matching

Factors:

- Distance
- Rating
- Acceptance Rate
- Completion Rate
- Response Time

2. Fraud Detection

Detect:

- Fake bookings
- Spam users
- Suspicious payments

3. Dynamic ETA Prediction

Use:

- worker location
 - historical jobs
 - traffic APIs
- ### 4. AI Customer Support Chatbot

Use:

- OpenAI API
 - RAG Architecture
 - Booking Context
-

OBSERVABILITY

Implement:

Logging:

- Winston

Monitoring:

- Prometheus
- Grafana

Error Tracking:

- Sentry

Audit Logs:

- All Admin Actions

TESTING

Coverage Target:

Minimum 85%

Implement:

Backend:

- Jest
- Supertest

Frontend:

- Vitest
- React Testing Library

E2E:

- Playwright

Test:

- Authentication
- Payments
- Booking Flow
- Chat
- Notifications

DEVOPS

Dockerize entire project.

Create:

Dockerfiles

docker-compose.yml

Infrastructure:

- AWS ECS
- AWS RDS (future)
- Mongo Atlas
- Redis
- CloudFront
- S3

CI/CD:

GitHub Actions

Pipeline:

Lint

→ Test

→ Build

→ Security Scan

→ Deploy

DOCUMENTATION

Generate:

1. Swagger/OpenAPI Documentation
2. Architecture Diagram
3. Database ER Diagram
4. API Reference
5. Deployment Guide
6. Postman Collection
7. Developer Onboarding Guide

EXECUTION STRATEGY

Work in phases.

For every phase:

1. Analyze existing code.

2. Identify affected files.
3. Explain required changes.
4. Implement code.
5. Add tests.
6. Update documentation.
7. Verify backward compatibility.

Never skip phases.

Always provide production-grade implementations.

When multiple approaches exist:

- compare them
- choose the most scalable
- explain why

Act as the technical owner of the platform and deliver enterprise-quality code.